Clean Air Finder - Understand it, demonstrate it, explain it

> **The whole project in one sentence:** a background collector saves real air measurements locally; the website compares that saved history; the Windows package brings its own Python, database engine and monitoring.

This is a learning guide, not a script to memorize. Read the short explanations first. Open the expandable reference boxes only when you need them.

Best reading experience: in VS Code, press **Ctrl+Shift+V** to open Markdown Preview. The reference boxes are collapsed there, so you do not face the whole document at once.

Choose your reading route

If you have...	Read...	You should be able to explain...
5 minutes	Sections 1-3	What it does, why these choices, why installation is unnecessary.
15 more minutes	Sections 4-6	Where data comes from, how the winner is calculated, every application control.
A technical interview	Sections 7-10	Tables, SQL, failure handling, trade-offs and a practical demonstration.

Contents

1. The one-minute explanation
2. How we developed it and why
3. Why unzip-and-run works, including SQLite
4. What happens end to end
5. The complete application UI
6. Useful feature combinations
7. Understand the database
8. Your SQL demonstration lab
9. Interview questions and short answers
10. Demonstrate it and be honest about its limits

Commands assume PowerShell is open in the project root. No application code or measurement data was changed to create this guide.

---

1. The one-minute explanation

What problem am I solving?

A family considering relocation wants to compare Bulgarian cities over a chosen historical period, using actual measurements rather than invented data.

The user chooses dates, PM2.5 or PM10, and a daily reference. The application finds the **lowest average measured pollution among cities with enough data**. It also explains coverage, stations, exclusions and limitations.

Five facts to remember

1. **Collecting and searching are separate.** Searching never calls the external provider.
2. **Lower concentration wins.** We do not invent a weighted multi-pollutant score.
3. **A winner is not automatically healthy.** It can still exceed the selected reference.
4. **Missing data is not zero pollution.** Coverage determines whether a station qualifies.
5. **Portable means bundled, not dependency-free.** The dependencies are already in the ZIP.

A short interview introduction

> "This is a local-first air-quality comparison service. It imports official EEA measurements into SQLite and builds daily aggregates. The user selects a historical period and pollutant; the API ranks eligible cities by their average concentration and explains data quality. I chose a portable Windows delivery so a reviewer can extract it and run one launcher without installing Python, a database server or Docker."

Be accurate about your role: development was AI-assisted. Explain the choices you approved, the implementation you now understand and the checks performed. Do not claim an independent human review or that you personally typed every line. The record is in AI_USAGE.md and PROMPT_HISTORY.md.

2. How we developed it and why

The order was: **understand -> verify real data -> build the smallest complete flow -> add reliability -> package -> test -> publish**.

▶ **Open the development story and decision/trade-off ledger**

3. Why unzip-and-run works, including SQLite

What the assignment required versus what we chose

The assignment required **easy deployment and one documented run command**. It did not mandate Docker or a particular cloud.

The owner additionally wanted a folder that could be zipped and handed to colleagues. Therefore, we chose **a complete Windows package**, not a source checkout that expects the reviewer to install a toolchain.

What is actually bundled?

Component	What it does
Start-CleanAir.cmd	Finds the bundled interpreter using the launcher's own location.
Python executable and Python DLL	Run Python independently of the user's installed Python.
Standard-library ZIP	Contains Python's standard-library modules.

Component	What it does
Bundled packages	FastAPI, Uvicorn, HTTPX, Jinja2, PyArrow, Pydantic, monitoring client and test tools.
Prometheus	Runs the monitoring service without Docker or a cloud account.
Data directory	Contains the separate, persistent measurement database.

The delivered runtime is CPython 3.13.16. The original machine's Python 3.12.10 was checked and used for building; it was not replaced.

How can moving the folder still work?

The launcher uses its own directory, application paths come from the source file's location, and python313._pth contains relative search paths:

```
python313.zip
.
..\packages
..\..
import site
```

That tells embedded Python where to find its library, packages and the project. There is no copied virtual environment depending on an old user's absolute path. Startup does **not** run pip or download Python. Normal users do not need pip. A developer rebuilding the runtime uses an installed Python/pip to prepare the bundled packages ahead of distribution.

The builder verifies official binary checksums and hash-pinned wheels. Exact versions and notices are in the runtime manifest and third-party documentation.

Is SQLite part of Python, or separate?

Both descriptions refer to different layers:

```
Python application
       |
       v
sqlite3                 Python standard-library API
       |
       v
_sqlite3.pyd            Compiled bridge into the engine
       |
       v
sqlite3.dll             SQLite database engine, bundled here
       |
       v
clean_air.sqlite3       Separate file containing our actual data
```

The actual data file is var/data/clean_air.sqlite3. The bundled engine is sqlite3.dll, currently SQLite 3.50.4; _sqlite3.pyd is Python's bridge.

Question	Answer
Do I install a separate database server?	No. The engine runs inside the Python process.
Do I run `pip install sqlite3`?	No. The Python interface is in the standard library.
Are the measurements inside Python's program files?	No. They live in the separate database file.
What survives a restart?	The database, backups and archives remain on disk.
Does each search create a new database?	No. Connections open the same persistent file.
Why do WAL/SHM files sometimes appear?	They support SQLite transaction/concurrency handling; they are not extra databases.
Can I copy an open database file by itself?	Do not. Use the backup API or stop/checkpoint it first.

Remember: Python includes access to SQLite. It does not include your application's measurement data; the application collects and stores that data.

The two download routes

- **Release ZIP:** extract and run. No Python, Docker, SQL-server or Git LFS installation is needed.
- **Git checkout:** Git LFS must retrieve the large files first. A small LFS pointer is not an executable.

Windows 10/11 x64, a writable local folder and a browser are assumed. Corporate device policy may require approval. Network/synchronized storage is unsuitable for this live WAL database. This is not an installed Windows service.

4. What happens end to end

A. Startup: from double-click to a working page

1. The launcher invokes bundled Python with `-m clean_air serve --open-browser`.
2. The CLI reads TOML settings, validates them and prepares workspace-local paths/logs.
3. SQLite schema version and WAL mode are checked/initialized.
4. A workspace lock prevents another application/manual writer using this installation.
5. Bundled Prometheus starts and must become ready.
6. FastAPI/Uvicorn starts; a small shutdown-control channel records the server identity.
7. The API lifespan starts one collector if online collection is enabled.
8. The browser opens after the live endpoint responds.

With an existing database, stored history is usable immediately. With an empty database, the page explains backfill progress; it does not invent a winner.

```
Browser ---- local HTTP ----> FastAPI ---- SQL ----> SQLite
                                  ^
                                  |
                            one collector thread
                                  |
                                  v
                        EEA metadata and Parquet

Prometheus ---- scrapes /metrics ----> application
```

B. Collection: how measurements reach disk

Think of this as an independent supply chain: discover, download, validate, store, summarize and checkpoint.

▶ **Open the collection pipeline step by step**

▶ **Source details an interviewer may ask about**

C. Searching: no external call in the user's request

1. Validate dates, pollutant, profile and custom threshold.
2. Check that initial history exists and the requested dates are stored.
3. Read indexed daily aggregates from local SQLite.
4. Select stations that meet the period's coverage policy.
5. Calculate city daily means, then period means.
6. Rank the unrounded means and return all tied winners.
7. Add reference statistics, exclusions, source/freshness and method details.
8. Render HTML, or return the same comparison as JSON.

With JavaScript, the form requests an HTML results fragment from `/results`. Without JavaScript, the form's ordinary GET submission still returns a complete page. The JSON route is `/api/cleanest`. All use the same comparison function.

D. The calculation, without mystery

Hourly-derived station day

- A UTC day has 24 expected hours.
- Default minimum: 18 valid distinct hours, or 75%.
- Mean = sum of observed valid values / observed valid hours.
- Missing hours are not zero.

Provider-reported daily value

- Prefer a valid reported daily mean when available.
- Preserve its reported capture percentage if supplied.
- If underlying hourly capture is unspecified, keep it unknown (`NULL`), not 100%.

Physical station and source precedence

- Multiple sampling methods at the same station do not receive duplicate city weight.
- For the same observation key, validated E1a wins over provisional E2a.
- Within that source class, the latest source result timestamp wins.
- An invalidating revision can remove a formerly valid value from the daily calculation.
- The source Verification flag is stored, but dataset priority is a separate rule.

Period

```
qualifying station means -> city mean for each day -> mean of city daily means
```

A station needs at least `ceil(number_of_requested_days * 0.90)` qualifying days. That is **27 days for a 30-day request**, or **329 for a 365-day request**. The city must also meet the period coverage requirement.

Illustration only	Day 1 station values	Day 2 station values	Period mean
City Alpha	10 and 30 -> city mean 20	20 and 40 -> city mean 30	25
City Beta	26 -> city mean 26	26 -> city mean 26	26

Alpha wins. It is still above a PM2.5 daily reference of 15 on both days. Changing the reference to 30 changes the descriptive within-reference count, not the winning mean. These numbers illustrate mathematics, not mock runtime data.

Default WHO daily references are **15 ug/m3 for PM2.5** and **45 ug/m3 for PM10**. The annual guidelines, 5 and 15 respectively, are not used to classify arbitrary periods. The European AQI's hourly colour bands are deliberately not applied to these period averages.

▶ **Precision, policy and reproducibility**

E. Failure, shutdown and retention

- Requests have bounded timeouts and retries for transport errors, 429 and 5xx responses. Defaults are a 90-second timeout and three attempts total, with bounded exponential delays between retryable attempts.
- Importing is **atomic per file**, not one transaction across the entire country. A partially failed refresh can expose new successful files alongside older ones.
- Failed files retain their last committed state and are replayed safely. Completed checkpoints survive a restart.
- Before any initial successful import, comparisons return an explicit not-ready response. Afterward, a provider failure does not remove the stored history.
- Collection success and measurement freshness are different; source age is monitored separately.
- The stop launcher requests cooperative shutdown, then handles leftover project processes, checkpoints SQLite and checks critical file locks. It does not indiscriminately kill Python.
- Records older than 36 months are verified into gzip JSON-lines archives before active copies are removed. Restoration verifies checksum/counts and rebuilds daily values. Deleting rows alone would not necessarily shrink SQLite's file; incremental vacuum and WAL checkpointing reclaim available storage. Restoration remaps source-file IDs by URL and checks foreign keys.

- **Known limitation:** automatic archiving runs after a successful collection cycle. Prolonged collection failures can delay it; a separate retention schedule is future work.

5. The complete application UI

This covers every application-owned navigation/form control. Cards and table headings that are only displays are identified so you do not invent features. External EEA/license websites are not application screens.

▶ **Main page: every link, field, button, card and expandable panel**
▶ **API page: every link and what each endpoint shows**
▶ **Monitoring: what to use in Prometheus**

6. Useful feature combinations

The quickest experiment is to keep dates/pollutant fixed and change only the daily reference. Then change the pollutant. This separates **reference statistics** from **eligibility/ranking** in your mind.

▶ **Open the feature/setup combinations you can demonstrate**

Why did I only see three cities?

At the checked snapshot, all 17 named cities had stored measurements. PM2.5 had some coverage-qualified history for seven cities, but only **Sofia, Varna and Ruse** qualified for the checked recent thirty days and year. PM10 qualified fourteen for the checked thirty days and thirteen for the year.

These are snapshot-specific counts, not a hard-coded three-city limit. The WHO threshold does not filter the eligible-city list.

Configuration versus search controls

- **UI:** dates, pollutant, WHO/custom profile and custom daily threshold.
- **TOML + restart:** ports, online collection, interval, initial 12-36 month window, coverage policy, freshness warning and monitoring retention.
- **CLI:** offline/no-browser/application-only diagnostics, manual sync, backup, archive/restore, status, tests, benchmarks and handover.

See settings and reference profiles. Coverage defaults are engineering choices, not a statement of legal compliance. Keep profile names/versions/source claims honest if editing built-in references.

Not implemented: a city-picker/map, live-now dashboard, user accounts, weighted multi-pollutant AQI, manual upload UI, or a CSV-download button. The SQL/export recipes below are operator techniques, not hidden UI features.

7. Understand the database

The relationship picture

```
cities
  |
  +-- stations
```

```
          |
          +-- sampling_points ---- observations ---- import_files
          |
          +-- station_days

metadata          application state
collection_runs   import progress/history
archives          verified cold-storage index
```

The nine application tables

Counts below are a checked snapshot from **2026-10-17 15:05 UTC**, not live promises.

▶ **Open the nine-table purpose/key/count map**
▶ **All columns and how to explain them**

8. Your SQL demonstration lab

Open a safe copy, with no extra database software

Use the project's own backup command, then Python's bundled SQLite console:

```
.\runtime\python\python.exe -m clean_air backup
if ($LASTEXITCODE -ne 0) { throw "Backup failed; do not continue." }
$demo = Get-ChildItem -LiteralPath .\var\backups -Filter 'clean-air-*.sqlite3' |
Sort-Object LastWriteTime -Descending | Select-Object -First 1
if ($null -eq $demo) { throw "No verified backup found." }
.\runtime\python\python.exe -m sqlite3 $demo.FullName
```

Now paste **SQL**, not PowerShell, into the SQLite console. Finish a statement with `;`. Type `.quit` to leave it.

This is Python's small SQLite console, not the full separate `sqlite3.exe` tool: do not assume `.tables`, `.schema`, `.headers` or `.mode` exist. Use the SQL and PRAGMA examples below instead. Tuple-style printed rows are normal.

Use SELECT/read-only PRAGMAs for the interview. The demo database is a copy; do not experiment with UPDATE/DELETE/DROP on the live tracked database.

▶ **Q1-Q2: show all tables and the amount of stored data**
▶ **Q3-Q5: raw data, a selected city, and useful filters**
▶ **Q6-Q8: cities, normalized days and why a city is excluded**
▶ **Q9: reproduce the core city-ranking calculation in SQL**
▶ **Q10-Q14: quality, collection, source files, freshness and archives**
▶ **Q15-Q18: unmapped stations, capture quality, schema and date gaps**

Change filters confidently

▶ **Open the SQL filter cheat sheet**

Show all data without flooding the terminal

The complete raw query is `SELECT * FROM observations ORDER BY started;`. For every row, use a CSV rather than scrolling through hundreds of thousands of tuples. The following uses a **read-only connection** to the live file and streams rows. It creates/overwrites only the export file.

▶ **Copy-and-run PowerShell: export all stored observations**

Application-code safety: SQL parameters use bound values, not concatenated user input. For example, the application passes dates and thresholds separately from its SQL text. Literal strings above are for your controlled console demo.

9. Interview questions and short answers

Use these to test understanding, not to recite words you cannot defend.

▶ **Deployment and architecture**
▶ **Data source and meaning**
▶ **Database and calculations**
▶ **Reliability, performance and ownership**

10. Demonstrate it and be honest about its limits

A ten-minute demonstration

1. Extract the portable package; show that its runtime and data are included.
2. Start with `--offline` for a predictable demo, leaving monitoring enabled.
3. Show the status/history range and explain UTC/completed days.
4. Compare PM10 over a useful stored period; explain winner, average and coverage.
5. Change only the reference threshold; show why the winner does not change.
6. Switch to PM2.5 and open exclusions; explain the smaller comparison scope.
7. Open the API result and monitoring; distinguish application health from freshness.
8. Open a verified SQL backup; show Q1, Q2, Q4 and Q9.
9. Explain a failure case, the archive policy and the known limits below.
10. Quit the SQL console, then use the stop launcher before moving/zipping files.

Operator command card

▶ **Open the complete run/stop/backup/sync/archive/test command card**

Limits to state without being defensive

- We compare **eligible measured cities**, not every Bulgarian city or personal exposure.
- Station types are not filtered to background-only; station locations and gaps matter.
- The checked year has coverage gaps; some requests correctly return insufficient data.
- Hourly updates require the app to be running and the provider to report.
- The current UTC day is stored as it arrives but not exposed as a completed-day result.
- Automatic archiving can be delayed by failed collection. Manual archiving exists.
- Monitoring has local alert states but no external pager/email delivery.
- The bundled runtime targets Windows x64 and the deployment is single-host/local-only.

- Benchmarks cover the documented dataset/machine/load, not every future scale.
- Most recent complete project validation had 69 tests passing; one upstream TestClient deprecation warning is documented, not hidden.
- The initial portable Release and later complete Git LFS checkout were both relocation/download tested on the same Windows machine, not a clean VM.

What to open if they ask "show me the code"

▶ **Open the source-file responsibility map**

Further evidence: methodology, operations, verification, AI usage and your exact prompt history.

Final self-test: explain the difference between source observations, station-day aggregates, city daily means and period means without looking. If you can then demonstrate the SQL and explain an excluded city, you understand the central design rather than just its presentation.